

```
In [ ]: #Load Libraries
import numpy as np
import sys
import matplotlib.pyplot as plt
import matplotlib.cm as cm
plt.style.use('dark_background')
%matplotlib inline
import tensorflow as tf

tf.__version__
```

Out[]: '2.14.0'

```
In [ ]: #Generate artificial data with 5 samples, 4 features per sample and 3 output cla
num_samples = 5
num_features = 4
num_labels = 3

#Data matrix (each column = single example)
X = np.random.choice(np.arange(3, 10), size=(num_features, num_samples), replace

#Class Labels
y = np.random.choice([0, 1, 2], size=num_samples, replace=True)

print("Original Data:")
print(X)
print("-----")
print(y)
print("-----")

# One hot encode class labels
y = tf.keras.utils.to_categorical(y, )
print("One-hot encoded labels:")
print(y)
```

Original Data:

```
[[6 4 7 6 7]
 [9 7 8 3 3]
 [6 8 9 7 4]
 [3 3 6 8 8]]
```

```
-----
[0 1 2 0 1]
-----
```

One-hot encoded labels:

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]
 [1. 0. 0.]
 [0. 1. 0.]]
```

A generic layer class with forward and backward methods

```
In [ ]: class Layer:
    def __init__(self):
        self.input = None
        self.output = None

    def forward(self, input):
```

```

    pass

    def backward(self, output_gradient, learning_rate):
        pass

```

```

In [ ]: ## Softmax activation class
class Softmax(Layer):
    def forward(self, input):
        self.output = np.array(tf.nn.softmax(input))

    def backward(self, output_gradient, learning_rate):
        return(np.dot((np.identity(np.size(self.output))-self.output.T) * self.output

```

```

In [ ]: ## Define the Loss function and its gradient
def cce(y, yhat):
    return(-np.sum(y*np.log(yhat)))

def cce_gradient(y, yhat):
    return(-y/yhat)

# TensorFlow in-built function for categorical crossentropy loss
#cce = tf.keras.losses.CategoricalCrossentropy()

```

```

In [ ]: # Step-1: add the bias feature to all the samples
x = np.concatenate((np.ones((1, X.shape[1])), X), axis=0)

# Step-2: initialize the entries of the weights matrix randomly
W = np.random.randn(num_labels, num_features + 1)

# Step-3: create softmax layer object softmax
softmax = Softmax()

```

```

In [ ]: # Step-4: run over each sample
for i in range(X.shape[1]):

    # Step-5: forward step
    # (a) Raw scores z = Wx = np.dot(W, x[:, i])
    z = np.dot(W, x[:, i])

    # (b) Softmax activation: softmax.forward(z)
    softmax.forward(z)

    # (c) Calculate cce loss for sample: cce(y[i, :], softmax.output)
    loss = cce(y[i, :], softmax.output)

    # (d) Print cce Loss
    print(f"Sample {i + 1} Loss: {loss}")

    # Step-6: backward step
    # (a) Calculate the gradient of the sample Loss w.r.t. input of the softmax
    output_gradient = cce_gradient(y[i, :], softmax.output)
    gradient = softmax.backward(output_gradient, learning_rate=0.1)

    # (d) Print gradient
    print(f"Gradient for Sample {i + 1}:\n{gradient}")

```

```
Sample 1 Loss: 26.41014419002036
Gradient for Sample 1:
[-1.00000000e+00  3.39015935e-12  3.39015935e-12]
Sample 2 Loss: 6.4175245323959734e-09
Gradient for Sample 2:
[ 9.99999994e-01 -6.41752451e-09  9.99999994e-01]
Sample 3 Loss: 23.97446015584045
Gradient for Sample 3:
[ 3.87279267e-11  3.87279267e-11 -1.00000000e+00]
Sample 4 Loss: 10.873010703724344
Gradient for Sample 4:
[-9.99981037e-01  1.89631923e-05  1.89631923e-05]
Sample 5 Loss: 0.00013474500869095827
Gradient for Sample 5:
[ 9.99865264e-01 -1.34735931e-04  9.99865264e-01]
Gradient for Sample 5:
[ 9.99865264e-01 -1.34735931e-04  9.99865264e-01]
```